

Solving Longest Common Subsequence (LCS) using DP

Problem Statement

Given two strings X and Y , find the length of the longest subsequence present in both of them. A subsequence is a sequence that appears in the same relative order, but not necessarily contiguous.

Example:

- $X = \text{ABCBADAB}$
- $Y = \text{BDCAB}$
- $\text{LCS} = \text{BCAB}$ or BDAB , length = 4

1. Recurrence Relation

We define a function $L(i, j)$ as the **length of the longest common subsequence** between the first i characters of string X and the first j characters of string Y .

The recurrence is:

$$L(i, j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ L(i-1, j-1) + 1 & \text{if } X[i] = Y[j] \\ \max(L(i-1, j), L(i, j-1)) & \text{if } X[i] \neq Y[j] \end{cases}$$

Intuitive Explanation

Case 1: Base Case

$$L(i, j) = 0 \quad \text{if } i = 0 \text{ or } j = 0$$

If either string is empty (i.e., has 0 characters), there can be **no common subsequence**, so the LCS length is 0.

Case 2: Last characters match

$$L(i, j) = L(i-1, j-1) + 1 \quad \text{if } X[i] = Y[j]$$

If the last characters of the substrings $X[0..i-1]$ and $Y[0..j-1]$ **match**, then:

- That character is part of the LCS.
- So, we add 1 to the result of the subproblem where we **ignore** this matching character (i.e., we look at $L(i-1, j-1)$).

Example:

$X = \text{"ABC"}, \quad Y = \text{"ADC"}$

When $i = 3, j = 3, X[2] = C, Y[2] = C \Rightarrow \text{match} \Rightarrow L(3, 3) = L(2, 2) + 1$

Case 3: Last characters do not match

$$L(i, j) = \max(L(i-1, j), L(i, j-1)) \quad \text{if } X[i] \neq Y[j]$$

If the last characters do **not** match, then:

- We **exclude one character at a time** from either X or Y and solve both subproblems.
- We take the **maximum** of those two solutions.

We're saying: *"Let's see whether dropping the last character from X or from Y gives us a better LCS."*

Recursion + Overlapping Subproblems = Dynamic Programming

The LCS relation is **recursive**, but with many overlapping subproblems (e.g., $L(i-1, j)$ appears in both cases 2 and 3). That's why it's ideal for memoization or tabulation.

Summary Table

Case	Condition	Meaning
Base	$i = 0$ or $j = 0$	No characters $\Rightarrow$ LCS = 0
Match	$X[i] = Y[j]$	Character included $\Rightarrow$ 1+ previous
No Match	$X[i] \neq Y[j]$	Try dropping one character and take max

2. Recursive Code (Naive)

```
def lcs_recursive(X, Y, i, j):
    if i == 0 or j == 0:
        return 0
    if X[i-1] == Y[j-1]:
        return 1 + lcs_recursive(X, Y, i-1, j-1)
    else:
        return max(lcs_recursive(X, Y, i-1, j), lcs_recursive(X, Y, i, j-1))
```

3. Memoization (Top-Down DP)

```
def lcs_memo(X, Y, i=None, j=None, memo=None):
    if i is None or j is None:
        i, j = len(X), len(Y)
        memo = [[-1] * (len(Y) + 1) for _ in range(len(X) + 1)]

    if i == 0 or j == 0:
        return 0
    if memo[i][j] != -1:
        return memo[i][j]

    if X[i-1] == Y[j-1]:
        memo[i][j] = 1 + lcs_memo(X, Y, i-1, j-1, memo)
    else:
        memo[i][j] = max(
            lcs_memo(X, Y, i-1, j, memo),
            lcs_memo(X, Y, i, j-1, memo)
        )
    return memo[i][j]
```

4. Tabular Version (Bottom-Up DP)

```
def lcs_tabular(X, Y):
    m, n = len(X), len(Y)
    dp = [[0]*(n+1) for _ in range(m+1)]

    for i in range(1, m+1):
        for j in range(1, n+1):
            if X[i-1] == Y[j-1]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

5. Illustrative Example

Let:

- $X = \text{ABCBADAB}$
- $Y = \text{BDCAB}$

Using the tabular method, we fill a $(m+1) \times (n+1)$ table:

		<i>B</i>	<i>D</i>	<i>C</i>	<i>A</i>	<i>B</i>
	0	0	0	0	0	0
<i>A</i>	0	0	0	0	1	1
<i>B</i>	0	1	1	1	1	2
<i>C</i>	0	1	1	2	2	2
<i>B</i>	0	1	1	2	2	3
<i>D</i>	0	1	2	2	2	3
<i>A</i>	0	1	2	2	3	3
<i>B</i>	0	1	2	2	3	4

The LCS length is in the bottom-right corner: 4

Conclusion

The LCS problem is a classic example of overlapping subproblems and optimal substructure — perfect for dynamic programming. While recursion is intuitive, memoization and tabulation are far more efficient.